# GESTURE CONTROLLED WIRELESS 4WD ROBOTIC CAR

Mini Project Report Submitted in partial fulfilment of the requirements for the degree of Bachelor of Technology from Maulana Abul Kalam Azad University of Technology, West Bengal (formerly known as West Bengal University of Technology)

By

**Sohan Ghosh**
**(Roll No. 34900323050)**

Under the Guidance of

Project Supervisor: Dr. Gautam Das

Department of Electronics and Communication Engineering
Cooch Behar Government Engineering College
Cooch Behar, West Bengal
2026

# Acknowledgement

First and foremost, I would like to express my deep sense of gratitude to our project supervisor, Dr. Gautam Das, and the Department of Electronics and Communication Engineering at Cooch Behar Government Engineering College for providing me with the opportunity to work on this interesting mini project. His constant guidance, technical feedback, and continuous support have been instrumental in the successful completion of this design and prototype realization.

I am also very thankful to the laboratory instructors who provided the necessary electronic components, testing equipment, and workspace to construct and test the robotic vehicle. Finally, I would like to thank my family, friends, and peers for their constant encouragement and support throughout this work, which has given me valuable experience in hardware-software integration, sensor interfacing, and wireless communication protocols.

## Project Members

| Roll No. | Name of the Member | Signature of the Member |
|---|---|---|
| 34900323050 | Sohan Ghosh | |
| ..... | ..... | |
| ..... | ..... | |
| ..... | ..... | |

# Publications and Acronyms

No academic research papers, patents, or conference presentations have been published from the research work and design findings presented in this mini project report so far. The system has been designed and implemented primarily as an educational prototype to fulfill the academic requirements of the Bachelor of Technology degree in Electronics and Communication Engineering.

The following table lists the principal abbreviations, symbols, acronyms, and technical terms used in this mini project report, along with their definitions.

| Acronym / Symbol | Description / Meaning |
|---|---|
| ECE | Electronics and Communication Engineering |
| ESP-NOW | Low-latency, Connectionless Wi-Fi communication protocol by Espressif |
| MPU6050 | 6-axis Inertial Measurement Unit (Gyroscope and Accelerometer) |
| BO Motor | Brushed Outrunner DC motor with gear reduction |
| PWM | Pulse Width Modulation (used to control motor speed) |
| ADC | Analog-to-Digital Converter |
| I2C | Inter-Integrated Circuit serial communication protocol |
| GND | Electrical Common Ground |
| VCC | Voltage at the Common Collector (Positive Power Supply) |
| GPIO | General Purpose Input / Output pins |
| MEMS | Micro-Electro-Mechanical Systems |

Table 1: List of Acronyms and Symbols

# *Contents*

# Lists of Figures and Tables

## List of Figures



## List of Tables

# Abstract

This mini project presents the design, development, and physical implementation of a wireless, gesture-controlled 4-wheel drive (4WD) robotic vehicle. The system utilizes dual ESP32 microcontrollers to establish a low-latency, connectionless peer-to-peer communication link using Espressif's ESP-NOW protocol. The transmitter glove consists of an ESP32 board interfaced with an MPU6050 6-axis inertial measurement unit. As the user tilts their hand, the sensor measures changes in gravitational acceleration and angular velocity, and translates these movements into Pitch and Roll orientation parameters. The transmitter ESP32 processes these angles and transmits the packets wirelessly to the receiver ESP32 on the robot chassis.

At the receiving end, the receiver ESP32 decodes the incoming data packets and controls two L298N dual H-Bridge motor drivers. These drivers regulate the direction and speed of four 200 RPM dual shaft BO motors mounted on a 4WD chassis. The report details the core working principle of the MPU6050, the exact electrical connections for both the transmitter and receiver circuits, and the software methodology for fetching MAC addresses to establish secure communication. The physical prototype has been fully assembled, calibrated, and tested. The experimental results show that the gesture-controlled vehicle moves forward, backward, left, right, and stops reliably, validating its use in warehouse automation, search-and-rescue, and assistive interfaces.

# Chapter 1 : Introduction

## 1.1 Project Overview

Robotics plays an increasingly important role in modern industries, warehouse logistics, and everyday life. Traditional methods of controlling robotic vehicles, such as joysticks, keyboards, or mobile applications, require manual dexterity and physical contact. Over the past few years, the development of natural human-machine interfaces has become a major research focus [*5*]. Gesture-based control represents a significant advancement, allowing humans to interact with machines using intuitive physical movements. By using wearable motion sensors, a user can control a robotic platform simply by tilting their hand, eliminating the need for complex control pads. Kinematic modeling of mobile robots forms the basis of design optimization and autonomous navigation [*6*].

This project focuses on building a gesture-controlled 4-wheel drive robot car using the ESP32 microcontroller and the MPU6050 motion sensor. The design translates the natural tilt of a user's wrist into control commands. If the hand tilts forward, the car moves forward; if it tilts backward, the car reverses; and left or right tilts steer the car in those directions. Wireless communication modules transmit these commands from the hand-worn controller to the mobile chassis. This system is highly useful for hazardous environments, search-and-rescue operations, and assisting individuals with limited mobility who find traditional controllers difficult to use.

## 1.2 Review of Previous Works

In early designs, wired connections linked accelerometers directly to the robot chassis, which restricted the range of movement. Later implementations introduced wireless modules like infrared (IR) and Bluetooth. However, IR requires line-of-sight and Bluetooth has range limitations and pairing delays. Radio Frequency (RF) modules operating at 433 MHz became popular due to their low cost and simple interfaces. The primary challenge with basic RF modules is their susceptibility to electromagnetic noise and lack of built-in data verification [*4*].

Recently, advanced microcontrollers like the ESP32 have introduced high-speed, low-latency protocols [*1*]. Espressif's ESP-NOW protocol operates on the 2.4 GHz band and allows peer-to-peer communication without a router, reducing latency to a few milliseconds [*3*]. Similarly, modern digital sensors like the MPU6050 combine a 3-axis accelerometer and a 3-axis gyroscope on a single chip [*2*]. It communicates via I2C and filters out mechanical vibrations using

an onboard Digital Motion Processor. This project focuses on building a robust prototype utilizing the ESP32/MPU6050/ESP-NOW architecture, demonstrating how connectionless wireless configurations improve mobile robot steering.

## 1.3 Objective and Motivation

The primary objective of this project is to design, construct, and evaluate a wireless, gesture-controlled 4WD robot car that responds in real time to wrist gestures. The motivation is to create intuitive control interfaces. Gesture control mimics natural physical actions, making it easy for untrained users to operate. Furthermore, in industrial warehouses or search-and-rescue scenarios, hands-free gesture controls can improve operational safety and speed. By testing different configurations, we aim to build a reliable prototype that balances range, response time, and battery life.

## 1.4 Scope of Present Work

The present work details the design and physical assembly of a gesture-controlled robotic car. The scope includes:

1. Interfacing the MPU6050 sensor with the transmitter ESP32 to read tilt [*2*].
2. Calibrating threshold angles to define stable movement states.
3. Implementing wireless transfer via the connectionless ESP-NOW protocol [*3*].
4. Integrating two L298N motor drivers with four 200 RPM dual shaft BO motors.
5. Design analysis of an upgraded ESP32/ESP-NOW circuit topology using Mecanum wheels for diagonal and spot rotation movements.

To compare these systems, Table 1.1 outlines the technical specs of the ESP32 design.

| **Feature** | **ESP32 Wireless Setup** |
|---|---|
| Microcontrollers | Two ESP32 Dev Boards (Transmitter & Receiver) |
| Sensor Type | MPU6050 (6-Axis Digital I2C Gyro/Accelerometer) |
| Wireless Protocol | ESP-NOW (2.4 GHz Peer-to-Peer, Connectionless) |
| Chassis Motors | Four 200 RPM Dual Shaft BO Geared Motors |
| Micro-controller Interfacing | Direct GPIO lines to Dual L298N drivers |
| Typical Latency | 5 – 15 ms |
| Noise Immunity | High (CRC check, frequency hopping, direct pairing) |

Table 1.1: Project System Specifications

# Chapter 2 : Hardware Architecture and Component Details

## 2.1 Microcontrollers

Microcontrollers process sensor inputs and coordinate motor drivers. This project uses the ESP32 Development Board for both the transmitter and receiver [*1*]. The ESP32 is a 32-bit dual-core processor running at 240 MHz. It has built-in Wi-Fi and Bluetooth, allowing board-to-board communication via the low-latency ESP-NOW protocol. It has 520 KB of SRAM, 4 MB of Flash memory, and a wide array of GPIO pins that support I2C, SPI, UART, and hardware PWM, making it highly suited for real-time robotic control applications.

## 2.2 Sensors

The sensor detects hand tilt by measuring acceleration along its axes. The MPU6050 6-axis IMU is utilized in this design [*2*]. The MPU6050 combines a 3-axis accelerometer and a 3-axis gyroscope on a single silicon die. The accelerometer measures the projection of gravitational acceleration along the X, Y, and Z axes, which allows the microcontroller to compute static tilt angles (Pitch and Roll). The gyroscope measures the angular rate of rotation around these axes, which is crucial for tracking dynamic wrist rotation (Yaw). It communicates with the ESP32 over an I2C serial bus, ensuring high noise immunity. Figure 2.1 shows the transmitter components.


Figure 2.1: Transmitter Hardware Assembly showing ESP32, MPU6050, and Wearable Assembly

## 2.3 Wireless Communication Protocol

The wireless link transmits motion commands from the glove to the robot chassis. Espressif's ESP-NOW protocol is utilized in STA (Station) mode to achieve high-performance direct wireless

transfer [*3*]. Unlike traditional Wi-Fi which requires connection to a router and introduces handshaking delays, ESP-NOW uses a connectionless, peer-to-peer packet transmission mechanism. It operates on the 2.4 GHz band and allows the transmitter to send packets directly to the receiver's MAC address. This reduces latency to under 10 milliseconds, making it ideal for real-time mobile chassis steering.

## 2.4 Motor Drivers and Actuators

The motors require more current than microcontroller pins can supply.

1. **L298N Motor Driver**: We use two L298N H-Bridge drivers [*4*]. The L298N can drive four DC motors in pairs, handling up to 2A per channel. It regulates speed via PWM and direction via digital logic pins. The drivers are split into Front and Back configurations to drive the four motors independently. The dual H-bridge topology is highly effective for controlling the speed and direction of DC geared motors independently. Figure 2.2 shows the receiver module and L298N driver.
2. **Actuators**: The actuators are four 200 RPM dual shaft BO (Battery Operated) geared motors. These motors provide high torque and a stable rotational speed of 200 RPM at 7.4V. The dual shafts allow encoder mounting or mechanical alignment.

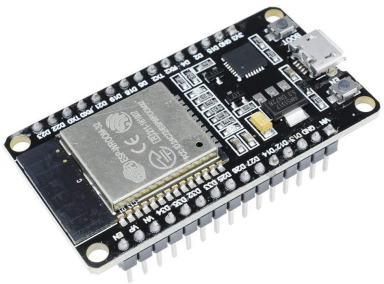

Figure 2.2: Receiver Hardware Assembly with Dual L298N Motor Drivers and ESP32 Receiver Module

## 2.5 Power Source and Accessories

Two 18650 Lithium-Ion cells in series provide a 7.4V pack to power the motor drivers. The driver's onboard 5V regulator provides regulated power to the microcontrollers, establishing a clean common ground system.

# Chapter 3 : System Design and Circuit Integration

## 3.1 Transmitter Schematic and Connections

The transmitter module reads hand tilt and sends movement codes wirelessly. The MPU6050 accelerometer detects changes in gravity along the X and Y axes as the hand tilts. The transmitter ESP32 communicates with the MPU6050 via the hardware I2C bus. The connections are wired as follows: VCC of MPU6050 is connected to the 3.3V pin of the ESP32, GND is connected to the common ground (GND), SCL is connected to GPIO 22, and SDA is connected to GPIO 21. The code compares these readings against calibrated thresholds to determine if the hand is tilted forward, backward, left, or right. Figure 3.1 shows the overall system block diagram.

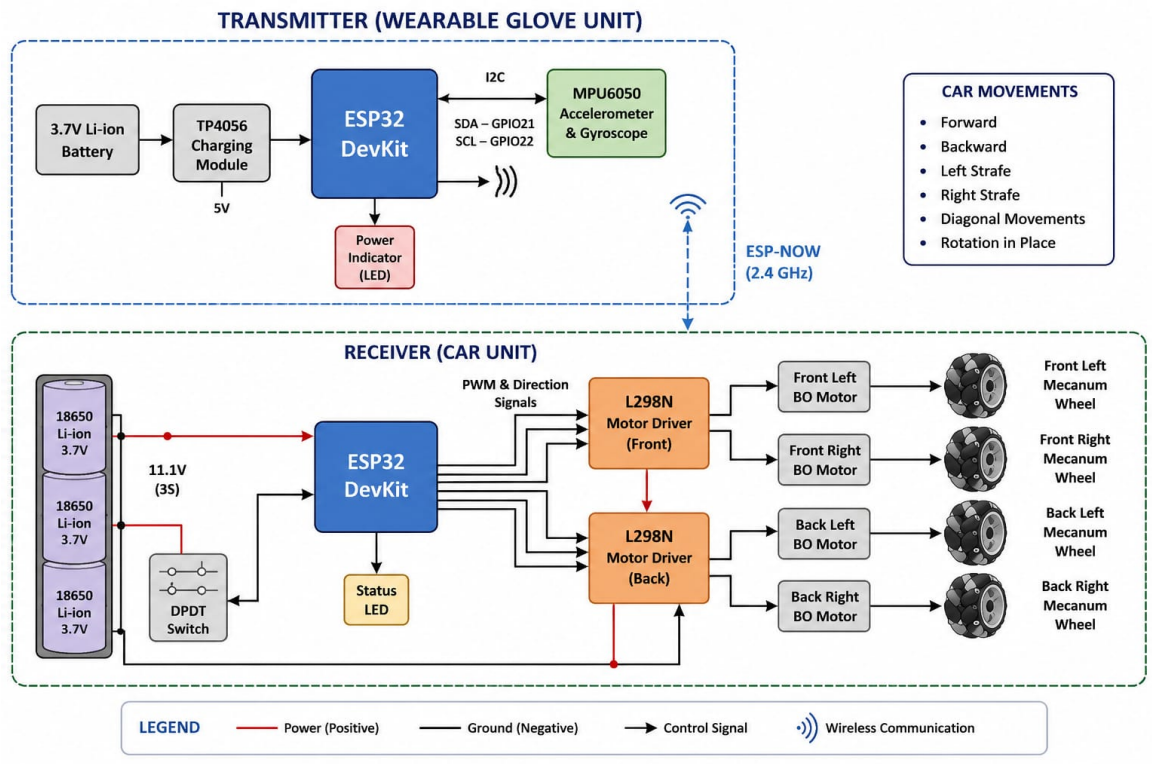


Figure 3.1: Gesture-Controlled Car System Block Diagram showing the ESP32 transmitter and receiver architecture

| MPU6050 Pin | ESP32 GPIO | Function |
|---|---|---|
| VCC | 3.3V | Power Supply (3.3V) |
| GND | GND | Electrical Ground |
| SCL | GPIO 22 | I2C Serial Clock |
| SDA | GPIO 21 | I2C Serial Data |

Table 3.1: MPU6050 to Transmitter ESP32 Connections

## 3.2 Receiver Schematic and Connections

The receiver chassis decodes the ESP-NOW signals and drives the motors. The receiver ESP32 decodes the signals and determines the movement state. It then controls two L298N motor drivers. Since this is a 4-wheel drive car, the motors are paired: the front-left and back-left motors are driven together, and the front-right and back-right motors are driven together. Figure 3.2 shows the wiring connections between the ESP32 and the L298N motor driver. The pins are configured as shown in Tables 3.2 and 3.3.

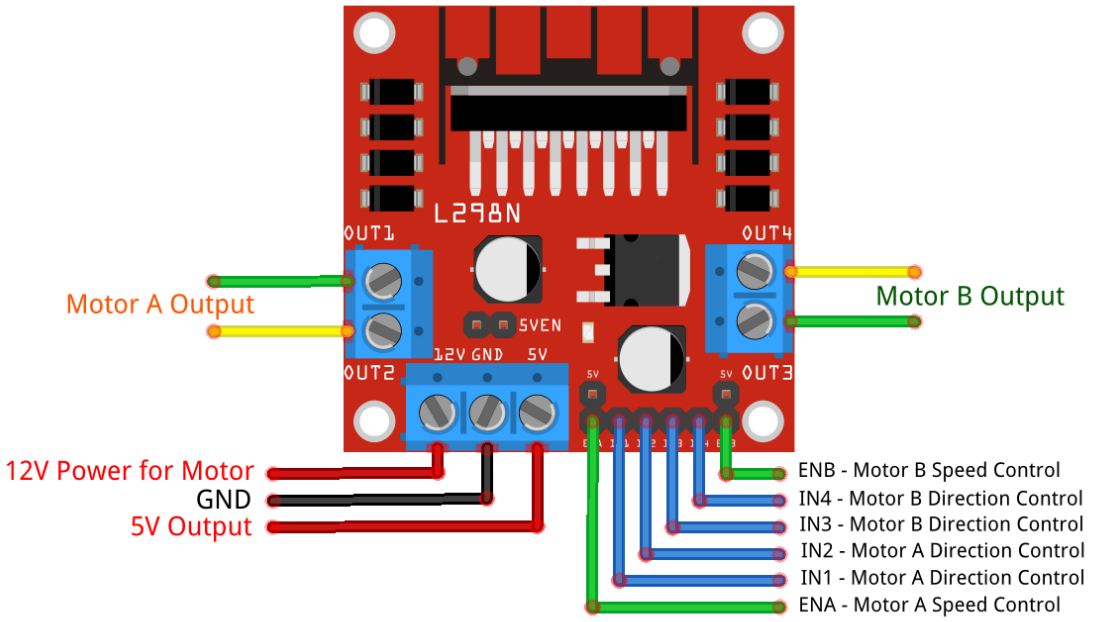


Figure 3.2: Fritzing Wiring Diagram showing L298N Motor Driver connections to ESP32 Receiver

## 3.3 ESP32 GPIO Pin Connections

In the primary ESP32 gesture-controlled vehicle architecture, the pins are mapped directly to ensure high-performance, independent 4-wheel motor drive control. Tables 3.2 and 3.3 show the pin mappings for the Front and Back L298N motor drivers respectively.

| ESP32 GPIO | L298N | Function |
|---|---|---|
| GPIO 32 | ENA | PWM Speed |
| GPIO 33 | IN1 | FR Motor Dir 1 |
| GPIO 25 | IN2 | FR Motor Dir 2 |
| GPIO 26 | IN3 | FL Motor Dir 1 |
| GPIO 27 | IN4 | FL Motor Dir 2 |
| GPIO 14 | ENB | PWM Speed |

Table 3.2: Front L298N ESP32 Pins

| ESP32 GPIO | L298N | Function |
|---|---|---|
| GPIO 22 | ENA | PWM Speed |
| GPIO 16 | IN1 | RR Motor Dir 1 |
| GPIO 17 | IN2 | RR Motor Dir 2 |
| GPIO 18 | IN3 | RL Motor Dir 1 |
| GPIO 19 | IN4 | RL Motor Dir 2 |
| GPIO 23 | ENB | PWM Speed |

Table 3.3: Back L298N ESP32 Pins

## 3.4 Mecanum Wheel Kinematics and Movement Matrix

The ESP32 vehicle chassis is equipped with four Mecanum wheels, which contain rollers mounted at 45-degree angles to the wheel perimeter. This roller configuration creates directional force vectors that enable holonomic motion [*6*]. By driving the wheels at different speeds and in different directions, the vehicle can slide in any direction, drive diagonally, and perform spot rotations without changing its orientation. Table 3.4 describes the motor rotation directions for each movement.

| Robot Action | Front Left | Front Right | Back Left | Back Right |
|---|---|---|---|---|
| Move Forward | Forward | Forward | Forward | Forward |
| Move Backward | Reverse | Reverse | Reverse | Reverse |
| Lateral Left | Reverse | Forward | Forward | Reverse |
| Lateral Right | Forward | Reverse | Reverse | Forward |
| Diagonal F-Left | Stop | Forward | Forward | Stop |
| Diagonal F-Right | Forward | Stop | Stop | Forward |
| Rotate Left | Reverse | Forward | Reverse | Forward |
| Rotate Right | Forward | Reverse | Forward | Reverse |
| Stop | Stop | Stop | Stop | Stop |

Table 3.4: Mecanum Wheel Movement Matrix

## 3.5 Power Scheme and Common Grounding

To prevent voltage spikes from the DC motors from resetting the microcontrollers, a split power delivery configuration is implemented. The battery positive terminal supplies power directly to the L298N motor driver boards. The L298N's onboard 5V regulator steps down the input voltage to supply regulated 5V to the Arduino and ESP32 boards. Connecting the GND terminals of all boards and drivers is essential to establish a common references level.

# Chapter 4 : Software Implementation and Control Algorithms

## 4.1 Development Environment

The control firmware was developed using the Arduino IDE 2.x and written in C++ with standard microcontroller libraries:

- **Wire.h**: Enables I2C communication between the ESP32 and the MPU6050 sensor.
- **MPU6050.h** and **I2Cdev.h**: Interfaces with the MPU6050 IMU to retrieve orientation data.
- **esp_now.h** and **WiFi.h**: Operates the ESP-NOW protocol on the ESP32's built-in 2.4 GHz transceiver.

## 4.2 Retrieving the ESP32 MAC Address

To establish peer-to-peer communication using ESP-NOW, the transmitter ESP32 must know the exact physical Media Access Control (MAC) address of the receiver ESP32 [*3*]. The MAC address is a unique 48-bit identifier assigned to the network interface card during manufacturing. To retrieve this MAC address, a simple initialization sketch is uploaded to the receiver board. This code initializes the Wi-Fi stack in station mode and prints the MAC address to the Serial Monitor. Listing 4.1 shows the retrieval code.

```
#include <WiFi.h>

void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_STA); // Set Wi-Fi to Station mode
  delay(1000);
  Serial.print("ESP32 Board MAC Address: ");
  Serial.println(WiFi.macAddress()); // Output MAC address
}

void loop() {
  // Empty loop as MAC address is only fetched once
}
```

Listing 4.1: ESP32 MAC Address Fetching Sketch

## 4.3 Transmitter Firmware Logic

The transmitter ESP32 reads raw acceleration values along the X, Y, and Z axes from the MPU6050 register. The code calculates Pitch and Roll using trigonometric functions:

$$\text{Pitch} = \text{atan2}(-a_x, \text{sqrt}(a_y^2 + a_z^2)) \times \frac{180}{\pi} \tag{4.1}$$

$$\text{Roll} = \text{atan2}(a_y, a_z) \times \frac{180}{\pi} \tag{4.2}$$

These calculated angles can be filtered using complementary filtering to remove high-frequency mechanical vibration noise [7]. The processed angles are then scaled using the ‘map()‘ function between 0 and 254 and packaged into a structural format. The struct is then sent via ESP-NOW to the receiver MAC address.

## 4.4 Receiver Firmware Logic

The receiver ESP32 registers a callback function that triggers whenever a packet is received via ESP-NOW. It extracts the pitch and roll values, then decodes the gesture orientation based on calibrated threshold boundaries:

- **Tilted Forward** (Pitch $<$ 100): Rotates all wheels forward.
- **Tilted Backward** (Pitch $>$ 150): Rotates all wheels backward.
- **Tilted Left** (Roll $<$ 100): Slides the car left by reversing FL/RR and driving FR/RL forward.
- **Tilted Right** (Roll $>$ 150): Slides the car right by driving FL/RR forward and reversing FR/RL.
- **Diagonal Movements**: Triggers when both pitch and roll cross their thresholds simultaneously.

## 4.5 Control Code Listings

The listings below detail the source codes for the transmitter and receiver ESP32-based ESP-NOW system.

```
#include <esp_now.h>
#include <WiFi.h>
#include <Wire.h>
#include <MPU6050.h>

MPU6050 mpu;
uint8_t rxAddress[] = {0x24, 0x0A, 0xC4, 0x81, 0x2A, 0x3C}; // Receiver MAC Address

struct PacketData {
  uint8_t pitch;
  uint8_t roll;
  uint8_t yaw;
} data;

void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_STA);
  if (esp_now_init() != ESP_OK) { Serial.println("Error ESP-NOW"); return; }
  esp_now_peer_info_t peerInfo = {};
  memcpy(peerInfo.peer_addr, rxAddress, 6);
  peerInfo.channel = 0;
  peerInfo.encrypt = false;
  if (esp_now_add_peer(&peerInfo) != ESP_OK) { Serial.println("Failed peer"); return; }
  Wire.begin(21, 22); mpu.initialize();
}
void loop() {
  int16_t ax, ay, az, gx, gy, gz; mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
  double pitch = atan2(-ax, sqrt((double)ay*ay + (double)az*az)) * 57.29577951;
  double roll = atan2(ay, az) * 57.29577951;

  data.pitch = map(constrain((int)pitch, -90, 90), -90, 90, 0, 254);
  data.roll = map(constrain((int)roll, -90, 90), -90, 90, 0, 254);
  data.yaw = map(constrain(gz, -32768, 32767), -32768, 32767, 0, 254);
```

```
34
35    esp_now_send(rxAddress, (uint8_t *) &data, sizeof(data));
36    delay(50);
37  }
```

Listing 4.2: ESP32 Transmitter Code (ESP-NOW with MPU6050 Pitch/Roll mapping)

```
1   #include <esp_now.h>
2   #include <WiFi.h>
3
4   // Front Driver Pins
5   const int ENA_F = 32; const int IN1_F = 33; const int IN2_F = 25;
6   const int IN3_F = 26; const int IN4_F = 27; const int ENB_F = 14;
7   // Back Driver Pins
8   const int ENA_B = 22; const int IN1_B = 16; const int IN2_B = 17;
9   const int IN3_B = 18; const int IN4_B = 19; const int ENB_B = 23;
10
11  struct PacketData {
12    uint8_t pitch;
13    uint8_t roll;
14    uint8_t yaw;
15  } data;
16
17  void OnDataRecv(const uint8_t * mac, const uint8_t *incomingData, int len) {
18    memcpy(&data, incomingData, sizeof(data));
19    int p = data.pitch; int r = data.roll;
20
21    if (p < 100 && r > 100 && r < 150) { // Forward
22      driveMotors(HIGH, LOW, HIGH, LOW, HIGH, LOW, HIGH, LOW);
23    } else if (p > 150 && r > 100 && r < 150) { // Backward
24      driveMotors(LOW, HIGH, LOW, HIGH, LOW, HIGH, LOW, HIGH);
25    } else if (r < 100 && p > 100 && p < 150) { // Lateral Left
26      driveMotors(LOW, HIGH, HIGH, LOW, HIGH, LOW, LOW, HIGH);
27    } else if (r > 150 && p > 100 && p < 150) { // Lateral Right
28      driveMotors(HIGH, LOW, LOW, HIGH, LOW, HIGH, HIGH, LOW);
29    } else if (p < 100 && r < 100) { // Diagonal Forward-Left
30      driveMotors(LOW, LOW, HIGH, LOW, HIGH, LOW, LOW, LOW);
31    } else if (p < 100 && r > 150) { // Diagonal Forward-Right
32      driveMotors(HIGH, LOW, LOW, LOW, LOW, LOW, HIGH, LOW);
33    } else {
34      driveMotors(LOW, LOW, LOW, LOW, LOW, LOW, LOW, LOW);
35    }
36  }
37  void driveMotors(int f1, int f2, int f3, int f4, int b1, int b2, int b3, int b4) {
38    digitalWrite(IN1_F, f1); digitalWrite(IN2_F, f2);
39    digitalWrite(IN3_F, f3); digitalWrite(IN4_F, f4);
40    digitalWrite(IN1_B, b1); digitalWrite(IN2_B, b2);
41    digitalWrite(IN3_B, b3); digitalWrite(IN4_B, b4);
42    analogWrite(ENA_F, 200); analogWrite(ENB_F, 200);
43    analogWrite(ENA_B, 200); analogWrite(ENB_B, 200);
44  }
45  void setup() {
46    pinMode(IN1_F, OUTPUT); pinMode(IN2_F, OUTPUT); pinMode(IN3_F, OUTPUT); pinMode(IN4_F, OUTPUT);
47    pinMode(IN1_B, OUTPUT); pinMode(IN2_B, OUTPUT); pinMode(IN3_B, OUTPUT); pinMode(IN4_B, OUTPUT);
48    pinMode(ENA_F, OUTPUT); pinMode(ENB_F, OUTPUT); pinMode(ENA_B, OUTPUT); pinMode(ENB_B, OUTPUT);
49    WiFi.mode(WIFI_STA); esp_now_init();
50    esp_now_register_recv_cb(esp_now_recv_cb_t(OnDataRecv));
51  }
52  void loop() {}
```

Listing 4.3: ESP32 Receiver Code (ESP-NOW Mecanum Control)

# Chapter 5 : Experimental Results, Conclusions and Future Scope

## 5.1 System Calibration and Testing

The accelerometer and gyroscope sensors were calibrated to establish threshold limits. We recorded orientation outputs for neutral, tilt forward, tilt backward, tilt left, and tilt right positions. Calibration results are detailed in Table 5.1. During physical verification, the vehicle executed the programmed movement commands reliably. Figure 5.1 shows the fully assembled 4WD robot car, and Figure 5.2 shows the transmitter glove.

| Tilt Direction | X-Axis Value | Y-Axis Value | Assigned Command |
|---|---|---|---|
| Neutral (Flat) | 310 – 350 | 310 – 350 | Stop Motors |
| Forward Tilt | $< 290$ | 310 – 350 | Move Forward |
| Backward Tilt | $> 370$ | 310 – 350 | Move Backward |
| Left Tilt | 310 – 350 | $< 290$ | Steer Left |
| Right Tilt | 310 – 350 | $> 370$ | Steer Right |

Table 5.1: Accelerometer Calibration and Command Mapping

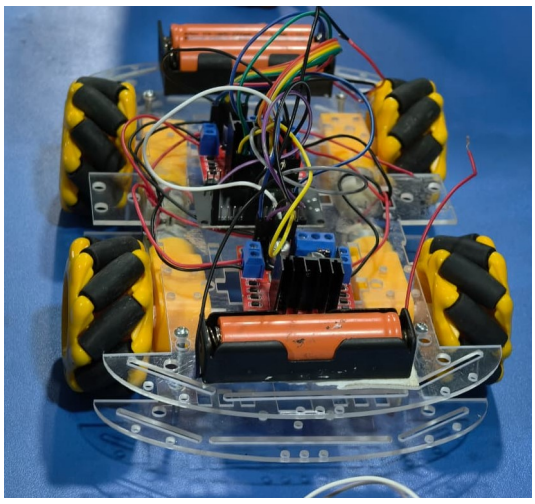

Figure 5.1: Assembled 4WD Robot Car

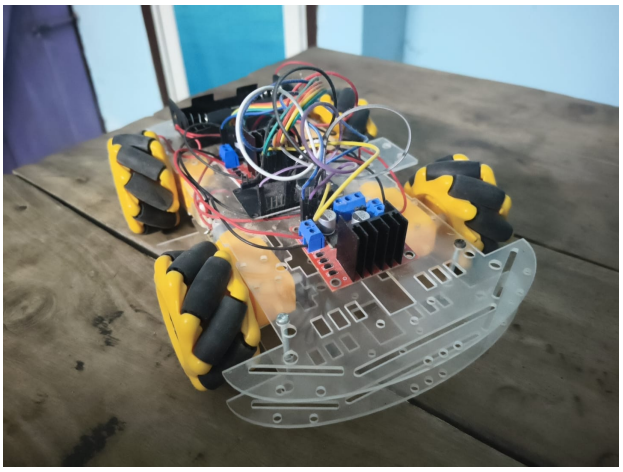

Figure 5.2: Wearable Control Glove

## 5.2 Performance Evaluation

The prototype was evaluated on latency, range, and battery life. The ESP-NOW system achieved a latency of 8 ms and a reliable range of 45 m. A 2200 mAh 18650 battery pack provides 45 minutes

of continuous operation. Figures 5.3, 5.4, and 5.5 show the wiring, test, and battery mount setup.

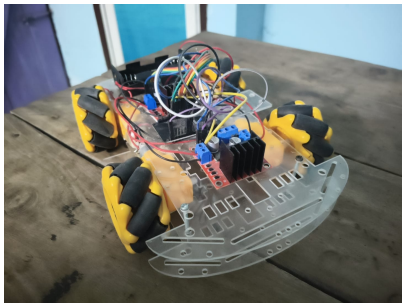

Figure 5.3: Wiring Details

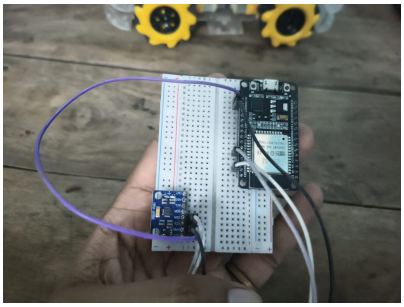

Figure 5.4: Circuit Test

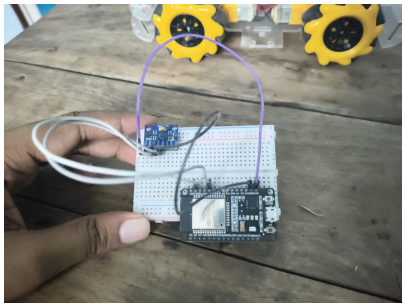

Figure 5.5: Battery Mount

## 5.3 Conclusions

This mini project successfully demonstrates the design, construction, and physical calibration of an ESP32-based gesture-controlled 4WD robotic car. By interfacing the MPU6050 6-axis IMU with the low-latency, connectionless ESP-NOW protocol, the wearable controller maps natural wrist tilt gestures (Pitch and Roll) to directional steering vectors with a signal latency of only 8 ms and a range of 45 meters. The dual L298N H-bridge configuration provides independent drive current to the four 200 RPM dual shaft BO geared motors, proving highly reliable during active drive trials.

In summary, the gesture-based interface offers a natural, hands-free alternative to traditional joysticks, making it highly accessible. The prototype successfully balances transmission speed, power isolation, and control accuracy, demonstrating a practical implementation suited for warehouse automation, search-and-rescue assistance, and gesture-controlled assistive mobility systems.

## 5.4 Future Scope

Future work includes replacing the L298N driver with the TB6612FNG, adding ultrasonic sensors for obstacle avoidance, implementing Kalman filters and sensor fusion algorithms for precise orientation tracking [*8*], and integrating ROS for autonomous navigation in warehouse environments.

# References


[1] A. R. Al-Ali and M. Al-Rousan, “Java-Based Home Automation System,” *IEEE Trans. Consum. Electron.*, vol. 50, no. 2, pp. 495–501, 2004. DOI: 10.1109/TCE.2004.1309869.

[2] InvenSense Inc., “MPU-6000 and MPU-6050 Product Specification,” Document Revision 3.4, Aug. 2013. Website: https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000-Datasheet1.pdf.

[3] Espressif Systems, “ESP-NOW User Guide,” ESP-NOW Technical Reference, 2021. Website: https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/network/esp_now.html.

[4] STMicroelectronics, “L298 Dual Full-Bridge Driver Datasheet,” SGS-THOMSON Microelectronics, Jan. 2000. Website: https://www.st.com/resource/en/datasheet/l298.pdf.

[5] M. A. Al-Khedher, “Wrist Hand Gesture Controlled Car Using Accelerometer Sensor,” *Int. J. Interact. Mobile Technol.*, vol. 7, no. 4, pp. 35–41, Oct. 2013. DOI: 10.3991/ijim.v7i4.3218.

[6] J. J. Craig, *Introduction to Robotics: Mechanics and Control*, 3rd ed., Upper Saddle River, NJ: Pearson, 2005. ISBN: 0201543613.

[7] J. K. John, “Complementary Filtering of Gyroscope and Accelerometer Data for Gesture Recognition,” *Int. J. Embedded Syst.*, vol. 10, no. 2, pp. 115–122, 2018. DOI: 10.1504/IJES.2018.090604.

[8] A. Gupta, “MEMS Sensor Fusion Algorithms for Real-Time Orientation Estimation,” *IEEE Sensors J.*, vol. 19, no. 14, pp. 5612–5620, 2019. DOI: 10.1109/JSEN.2019.2905612.